

LLM-Enhanced Process-Aware Knowledge Tracing from Student Solution Steps for Concept Deficiency Prediction

Temesgen
University of Vienna

June 2026
Updated 16 September 2026

Abstract

Most knowledge tracing (KT) systems only predict whether a student will get the next question right. The recent ConceptKT benchmark asks a harder and more useful question: which of 55 math concepts will a student be weak in on their next problem? Trained KT models do this very poorly, and only large language models (LLMs), used directly at prediction time, do reasonably well, but they are slow and costly to run. This report tests a middle path. We run a small, low-cost Gemini model once on each *past* solution, offline, to pull out a few simple signals: the concepts the student seems to have missed, the type of error, the faulty step, and a short summary of what happened. A small LSTM then learns from these past-only signals to predict the student’s future weak concepts. On the official MathEDU/ConceptKT split (3,644 train and 404 test interactions from 6 students), averaged over five random seeds, the LLM signals raise deficiency macro-F1 over 13 categories from 24.3% (a correctness-only baseline) to 27.4%. That covers about 40% of the observed gap to a gold-feature reference of 32.1%, and both precision and recall improve. We block label leakage with occurrence-specific feature keys and a runtime check that we test to make sure it actually fires, and the full run repeats offline with a fixed cache and software environment. The answer to the research question remains exploratory: the gain is not statistically significant, does not extend to leave-one-student-out evaluation. Full B does not exceed a mask-only baseline.

1 Introduction

Knowledge tracing tries to follow how a student’s understanding changes over time and to predict how they will do next. Classic methods look only at which questions were asked, the skills involved, and whether each answer was right or wrong [1]. This works well for predicting the next answer, but it throws away the student’s actual working, which often says far more about what they misunderstand than the final answer does [2].

Newer work has started to use the solution process itself, and to bring language models in for their semantic and reasoning signals [3, 6, 7]. The ConceptKT benchmark [5] pushes this further by changing the task. Instead of predicting a single right or wrong bit, it asks which specific concepts a student is currently weak in. This is useful, because it tells a teacher what to go back over, but it is also hard. On ConceptKT, OKT gets 1.87% deficiency macro-F1, while a full LLM reaches about 17%; several trained baselines have no reported deficiency score.

We study the question from the proposal:

Can LLM-derived process signals from student solution steps improve concept deficiency prediction compared with a correctness-only baseline?

The key idea is to use the LLM as an offline analyzer, not as the predictor. It reads each past solution once and writes down a few structured signals. A small trained model then uses those signals

to predict future weak concepts. This plays to each part’s strength: the LLM is good at reading one solution, and a tiny LSTM is a sensible choice when there are only six students. The work has four parts:

- A complete and repeatable pipeline (an offline Gemini analyzer with a disk cache, careful sequence building, and a two-head LSTM).
- A clean three-way comparison that separates the value of the LLM signals from everything else and also compares them with gold past features.
- An honest setup: leakage is blocked and tested, the class imbalance is handled directly, results are averaged over five seeds, and the whole thing reruns offline with the recorded cache and software environment.
- Ablations, paired tests, and per-student and leave-one-student-out checks.

The headline is simple. The offline LLM signals add 3.1 points of macro-F1@13 on average over the baseline and close about 40% of the observed gap to gold features. With six students and 59 positive test labels the exact decimals jump around, and neither the order, A below B below C, nor the precise values are statistically reliable across seeds. B also does not exceed the mask-only baseline (27.50%).

2 Related Work

Deep Knowledge Tracing brought neural sequence models to KT and set the standard way of predicting performance from a history of interactions [1]. Later models refined the backbone but still read only correctness and skill ids. A separate line argues that remembering a fact is not the same as applying it, and models the solving process for the sake of interpretability [2]. We share that motivation but aim at the harder concept-deficiency output instead of correctness.

Text and language models have also been folded into KT. Using language-model representations of question and concept text helps, especially for cold-start items where there is little history [3]. KCQRL shows that an LLM can label knowledge concepts at the level of each solution step, and that the resulting concept-aligned embeddings improve 15 different KT models on two large datasets [7]. That is the basis for two of our choices: getting concept and process signals from an LLM, and turning the LLM’s short summary into a fixed sentence embedding.

The closest method is StatusKT, which runs a three-stage LLM pipeline to read proficiency signals out of solving processes and feeds them into existing KT models, improving both accuracy and interpretability over correctness-only KT [6]. We follow the same idea, an LLM that produces structured signals for a trained KT model, but in a much smaller, low-resource form, and we measure the signals against a gold-feature reference. The task, label set, and split all come from ConceptKT [5], which is built on MathEDU, a set of real student solutions with correctness labels, teacher error notes, and feedback [4]. ConceptKT runs its LLM end-to-end at prediction time; the trained baselines it reports use only correctness. Our hybrid sits between those two.

3 Data

We use two public repositories from the NYCU NLP Lab that describe the same 4,048 interactions from 6 students, seen from two sides. MathEDU [4] is the process side: each record has a problem `id`, a `student_id`, the final answer, the worked solution in $\text{\LaTeX}$, a correctness label, and, when wrong, a teacher note with an error type and a faulty equation. ConceptKT [5] is the label side: for every record it gives the gold `associated_concepts` (what the problem needs) and `missing_concepts` (what the student failed to apply, empty when correct), from a fixed list of 55 concepts in 13 categories. It also ships the official 90/10 split in time order (3,644 train and 404 test).

We merge the two on the pair (`id`, `student_id`), which matches all 4,048 records with no gaps; `id` on its own is not unique because students share MathQA problems. ConceptKT acts as the backbone of the merge, since it owns the time order, the official split, and the gold labels, and MathEDU’s process fields hang off it. Two small things had to be handled so the split lines up exactly: two repeated (`id`,

Table 1: Eight raw error strings mapped to six types (including none).

Raw MathEDU string	Final type
Wrong mathematical operation/concept; Comprehension error	wrong_operation_or_concept
Lack of necessary mathematical concepts	lack_of_concepts
Unfinished answer	incomplete_answer
Arithmetical error; Algebraic error; Measurement error	calculation_error
Careless error	careless_error

Table 2: MathEDU/ConceptKT corpus statistics.

Quantity	Value	Quantity	Value
Interactions	4,048	Concepts / categories	55 / 13
Students	6	Correct answers	3,050 (75.3%)
Train / test (official)	3,644 / 404	Wrong answers	998 (24.7%)
Records with gold deficiency	463 (11.4%)	Concept mistakes	722 (17.8%)
Test positive concept labels	59	Careless errors	276 (6.8%)

`student_id`) keys with conflicting labels, which we resolve by taking the later copy for the test set, and MathEDU’s eight raw error strings, which we fold into a six-way set of error types. One point is easy to get wrong: the time order is the position inside ConceptKT, grouped by student. The `id` is a problem number, not a timestamp, so the data must never be sorted by it.

Feature vectors now use (`student_id`, `seq_pos`): the old problem key could overwrite earlier labels with later ones. For example, masked student S* has problem Q* at positions 573 (train, *Mixed Solution Concentration*) and 649 (test, *Basic Math Operations*); both are now kept separately. All results were rerun after this fix, in the recorded software environment. These conflicting annotations are not verified re-attempts. Table 1 gives the exact normalization; absent or unknown errors map to `none`.

Table 2 sums up the corpus, and three facts from it shaped the design. The data is very lopsided: 75.3% of answers are correct, and only 463 records (11.4%) carry a real deficiency target. The signal is also thin at test time, with just 59 positive concept labels spread across the 404 test targets, which makes macro-F1 noisy and is why we lean on imbalance-aware loss, threshold tuning, and a coarser 13-category view. Finally, MathEDU has no question text, so the analyzer works from the solution alone. That does not touch the deficiency targets, which are gold.

4 Method

Figure 1 shows the flow. Each raw record goes through the LLM analyzer, gets turned into numbers, is placed into a leakage-safe sequence, and is fed to a two-head LSTM whose deficiency head is limited to the concepts the target question actually needs.

4.1 The offline LLM analyzer

For each interaction the analyzer makes one call to Gemini (model `gemini-3.1-flash-lite`, taken from an environment variable so the model can be swapped without touching code). The prompt asks it to describe what already happened in the solution and to return a fixed JSON object: the associated concepts, the missing concepts, a six-way error type, the faulty step, a note on partial understanding, and a one or two sentence summary. A JSON schema forces the shape of the answer. A few choices keep this part cheap and reliable. The temperature is set to zero, which encourages consistent output; cached responses make reruns repeatable. Every result is saved to disk under a key built from the prompt and model, so re-running the pipeline makes no API calls at all and identical solutions share a file. The first extraction made 3,882 live calls with zero failures and produced 3,962 cached results that cover all 4,048 interactions. The corrected reruns use this cache, with no API calls. We picked a small, low-cost model on purpose, to keep the whole extraction cheap; a larger state-of-the-art LLM might give cleaner signals, but this was not tested. A system message keeps the model in line: it must pick

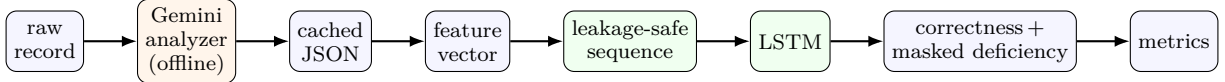


Figure 1: The full data flow. The LLM is called once per past interaction and its output is saved. Everything after it is a small model that runs the same way every time.

Table 3: Per-interaction feature parts and which arms use them. The associated concepts are known when the question is shown, so they also serve as the output mask.

Component	Dim	Used by	Source
assoc_multihot	55	A, B, C	gold associated concepts (and mask)
correct	1	A, B, C	grading
llm_missing_multihot	55	B	analyzer’s missing concepts
llm_error_type	emb.	B	analyzer’s error type
summary_embed	384	B	frozen encoding of the summary
gold_missing_multihot	55	C (and target)	gold missing concepts
gold_error_type	emb.	C	gold (normalized) error type

concepts only from the fixed 55, and for a correct answer it must leave the missing list empty and the error type as `none`. A small cleanup step afterwards drops any made-up concept and re-checks those rules, so a sloppy answer cannot corrupt the labels. Failures are split on purpose: a malformed JSON or schema error is retried once, then stored as an empty placeholder, while a real API or key error is retried once and then raised, never cached, so a bad key cannot poison the cache.

4.2 Turning signals into features

Each interaction becomes the components in Table 3. The error type is passed as a small id and learned as an 8-dimensional embedding inside the model, since a small category like this is better learned than hand-set. The summary is turned into a vector by a frozen `all-MiniLM-L6-v2` sentence encoder [8]; with six students there is no data to train an encoder, so we just use it as a fixed way to read meaning out of the text, much as KCQRL does [7].

The 384-dimensional summary is L2-normalized and concatenated directly, with no projection or activation. Blank summaries encode a single space; Arm C uses zeros. For a real masked example, the solution gives $9 < a \leq 21$, $19 < b < 31$, then wrongly concludes $21/19 \leq a/b < 31/9$. Its analyzer JSON includes `{"missing_concepts":["Range of Values"], "error_type":"wrong_operation_or_concept"}`. The associated mask and missing vector both activate index 3, correctness and error ID are 0, and the summary norm is 1.000. The full JSON, faulty step and summary are in `results/revisions/feature_example.json`. The B vector has 503 dimensions.

4.3 A leakage-safe sequence model

We build one sequence per student in time order. At step i the model sees the features of interaction i and predicts interaction $i+1$. Because the prediction for a target comes from the LSTM state after reading only the steps before it, a target’s own features never reach the model that predicts it. The backbone is a plain single-layer LSTM (hidden size 64, output dropout 0.4) with two heads on the next-step state. The correctness head is one output trained with cross-entropy. The deficiency head has 55 outputs, but we restrict predictions to the associated concepts and push the outputs for the rest down to a very large negative value before the loss and the prediction. This drops about 53 of 55 obvious negatives per example and lets the model focus on the real candidates. Those remaining outputs are trained with a focal version of cross-entropy [9], which lowers the weight of the many easy negatives. The model is kept small on purpose, because with six students anything bigger risks memorization; larger models were not tested.

For completed step i , $h_i = \text{LSTM}(x_i, h_{i-1})$ feeds a correctness head u_i and deficiency head $z_{i,c}$. Let $a_{i+1,c}$ be the next question’s associated-concept mask and $y_{i+1,c}$ its gold deficiency label. The masking

and loss over a student’s training targets I are

$$\tilde{z}_{i,c} = \begin{cases} z_{i,c} & a_{i+1,c} = 1, \\ -10^9 & a_{i+1,c} = 0, \end{cases} \quad p_{i,c} = \sigma(\tilde{z}_{i,c}), \quad p_{i,c}^* = y_{i+1,c}p_{i,c} + (1 - y_{i+1,c})(1 - p_{i,c}), \quad (1)$$

$$L = \text{BCE}(u, r) + \frac{\sum_{i \in I, c} a_{i+1,c} \max(1 - p_{i,c}^*, 10^{-6})^2 \text{BCE}(\tilde{z}_{i,c}, y_{i+1,c})}{\sum_{i \in I, c} a_{i+1,c}}. \quad (2)$$

BCE uses logits; correctness BCE is averaged over I . There are no additional class weights. Empty masks contribute zero; each student gives one optimizer step.

Six source records have missing labels outside this mask: four training, one validation and one test record. The loss ignores these positives; evaluation keeps them. Thus 58 of 59 test concept positives are reachable. We retain this constrained task and original labels; expanding masks from target labels would leak information. Excluding the one inconsistent test row gives F1@13 A/B/C = 24.28/27.39/32.07%, with no refitting. Full exceptions and diagnostics are in `report/Diagnostic-Supplement.md`.

4.4 The three arms

All three arms use the same LSTM and the same training code, and each one is trained separately from scratch. The only thing that changes is the per-step input, so the comparison tests different feature inputs under the same training procedure.

- **Arm A (baseline).** Associated concepts plus the correct/wrong bit. This is the correctness-only model: what you can predict from a pass/fail record with no LLM.
- **Arm B (process-aware).** Arm A plus the analyzer’s signals, that is, its missing concepts, the summary vector, and the learned error embedding. This is the system we are testing.
- **Arm C (gold reference).** Arm A plus the *gold* missing concepts and gold error type taken from the human labels, used only for past steps. These are gold past signals; C lacks summary features, so it is not a guaranteed upper bound.

The two comparisons that matter are B minus A, the value of the LLM signals over plain correctness, and C minus B, the difference between gold and analyzer features. A useful check falls out of the design: arms A and C never use LLM features, so their inputs are the same whether we run on synthetic features or the real Gemini ones. Only arm B moves, which is exactly what we want.

4.5 Blocking leakage on purpose

The rule is that only the target’s known associated concepts may be seen before predicting it, and the analyzer must never see the target’s own solution. We make this part of the structure rather than a habit. The analyzer only ever gets one finished interaction at a time. The model is causal by construction. The gold missing concepts are the target, and they are used as an input only by arm C and only for past steps. On top of that, a runtime check runs on every sequence and confirms that each target comes strictly after every input used to predict it. To be sure the check works, a small negative test feeds it a sequence with leakage on purpose and confirms the check fires. Using the associated concepts as an input and a mask is not leakage, since they are known the moment the question appears.

The index guard is:

```
for target, past in prediction_pairs:
    assert target not in past
    assert all(i < target for i in past)
```

Tests `test_assert_no_leakage_fires_when_target_in_history`, `test_duplicate_problem_ids_preserve_occurrence_labels`, and `test_target_and_future_features_cannot_change_current_prediction` guard ordering, occurrence identity and causal predictions, respectively.

5 Experimental Setup

We use the official split (3,644 train and 404 test). For early stopping we hold out the last 10% of each student’s training history as validation, still before any test target. Each arm and seed is trained with Adam (learning rate 10^{-3}) for up to 60 epochs, stopping early on validation loss with a patience of 8. The feature store is built once and shared across arms and seeds, and we report mean and standard deviation over five seeds.

The deficiency head needs one extra step. It is so imbalanced that a fixed 0.5 cutoff can produce mostly negative predictions, which would hide differences between arms. So after training we try a small grid of thresholds and keep the one that gives the best macro-F1 on validation, then apply it once to the test set. The cutoff is picked on validation and never on test. We score correctness with accuracy and AUC, and deficiency with macro precision, recall, and F1 at both the 55-concept level and the steadier 13-category level. With five fixed seeds, the official split, and a warm cache, training makes no API calls and supports reproduction under the recorded software environment. A set of 73 unit tests, none of which need the API, guards the taxonomy, the feature layout, the leakage check, the caching and cleanup, the metrics, and the model shapes.

For seeds 0–4, the threshold grid is $G = \{.10, .15, .20, .25, .30, .35, .40, .50\}$, and $\tau^* = \arg \max_{\tau \in G} \text{MacroF1}_{13}(Y_{val}, \mathbf{1}[P_{val} \geq \tau])$. Concept predictions are OR-aggregated into categories. Ties take the smallest threshold; no validation positives gives .50. The flow is validation loss $\rightarrow$ best checkpoint $\rightarrow$ validation F1 sweep $\rightarrow$ frozen threshold $\rightarrow$ test. A and B each select (.35, .35, .35, .35, .35); C selects (.30, .35, .35, .35, .35). The small hidden state and dropout limit capacity; Adam’s 10^{-3} rate is the original default choice. No hyperparameter search established these values as optimal. Keeping them fixed isolates feature changes. Artifacts: <https://github.com/temesgen-mulugeta/process-aware-kt>, branch `codex/advisor-revisions`, `code/artifact` commit `9a16cb6`. `results/revisions/` contains all 63 runs, seeds, predictions, thresholds, the exact run-source snapshot, source/encoder hashes and resolved environment. The cache is not distributed; saved-score verification is possible without it, but retraining requires it. See `report/ARTIFACTS.md`.

6 Results

Before trusting the analyzer on all 4,048 records, we checked it against the gold teacher notes on 20 wrong answers. It matched the gold error type on 15 of 20 (75%), with many exact faulty-step matches. The misses sit on the fuzzy line between a calculation error and a wrong-concept error, which is hard even for people, so we went ahead.

The expanded check covers all 87 wrong, annotated validation answers and gives $44/87 = 50.6\%$ agreement. Table 4 reports per-class precision/recall. On 45 gold-deficient records (49 concept labels), exact-concept micro precision is 27.0% and recall 20.4%, without an associated-concept filter. These conditional scores exclude gold-negative cases. The 75% spot check is not representative. Always predicting wrong operation/concept gives $54/87 = 62.1\%$ accuracy, above the analyzer; its six-class macro-F1 is 16.1%.

Table 4: Error-type agreement on all 87 annotated wrong validation answers. Undefined precision/recall is reported as zero.

Gold class	Support	Precision	Recall
Wrong operation/concept	54	0.695	0.759
Lack of concepts	11	0.000	0.000
Calculation	8	0.176	0.375
Incomplete answer	13	0.000	0.000
Careless	1	0.000	0.000
None	0	0.000	0.000

Table 5: Deficiency prediction on the official test set, mean and standard deviation over five seeds. Scores are percentages except AUC; SD is in percentage points. Correctness accuracy is flat because that head is swamped by the 75% rate of correct answers. Deficiency is where the process signals matter.

Arm	Acc.	AUC	F1@55	F1@13	P@13	R@13
A: correctness-only	71.78	0.602	10.12	24.28	20.41	39.81
B: process-aware	71.78	0.587	11.58	27.39	26.31	41.28
C: gold reference	71.78	0.595	13.70	32.06	29.73	48.88
Mask-only	n/a	n/a	15.20	27.50	18.24	84.62
std on F1@13				A: ± 3.92	B: ± 5.03	C: ± 1.94

6.1 Main result: do the LLM features help?

Table 5 has the main numbers, and the mean order is the same on every deficiency metric: A below B below C, but not on every seed.

A diagnostic added during review predicts every target-associated concept as missing, without training, history or an LLM. It scores 27.50% F1@13 and 15.20% F1@55: B does not exceed it at either level, and no main arm exceeds it at F1@55.

Three things stand out. The LLM signals help on average: arm B beats arm A by 3.1 points of macro-F1@13 (24.28 to 27.39) and 1.5 points at the concept level (10.12 to 11.58), and both precision and recall go up, but this does not rule out threshold effects. The signals also recover part of the observed gap to gold: the gap from A to C is 7.8 points, and arm B reaches 27.39, about 40% of the way, leaving 4.7 points that do not guarantee a gain from a sharper analyzer. And correctness stays flat at 71.78% across all arms, because that head is dominated by the base rate of correct answers. Correctness is simply not where these signals pay off; deficiency is, which is what ConceptKT found too [5].

The 95% seed-mean F1@13 intervals are A [18.84,29.71], B [20.41,34.36], C [29.38,34.75]%. These use $t_{.975,4s}/\sqrt{5}$, with sample SD s ; the tables retain the original population SD convention. Paired two-sided t tests give $p = .494$ (B–A) and $.185$ (C–B), or $.988$ and $.369$ after Bonferroni adjustment. Exact sign-flip tests on five seed pairs (32 sign assignments) give $.5625$ and $.25$; their minimum two-sided p is $.0625$. Neither gain is significant. The B–A 95% interval is $[-8.38, +14.61]$ points. This measures seed uncertainty on six fixed students, not new students.

Table 6 breaks the B minus A difference down by category. The signals help these multi-step and word-problem topics on average: Sequences and Series (+0.406), Physics (+0.032), and Statistics and Probability (+0.044). Their positive test counts are only 2, 6 and 8, so these gains do not establish why the features help. A few categories slip a little (Set Theory -0.044 , Finance -0.008 , Prime Numbers -0.026), possibly because the extra features add noise; positive counts are only 1, 8 and 2. Across the 13 categories, four improve, six stay the same, and three drop.

For Sequences and Series, mean (TP, FP, FN, TN) changes from (0.4, 1.4, 1.6, 400.6) in A to (1.2, 1.0, 0.8, 401.0) in B. Full counts for all 13 categories are in `results/revisions/summary.json`.

For context, Table 7 shows the numbers ConceptKT reports [5]. The trained KT models predict correctness fairly well (63 to 69%); OKT gets 1.87% on deficiency, while several others are unreported, and a full LLM reaches about 17%. This is context, not a fair race. We use the same split and gold labels, but our setup is different (a small model on past-only offline features against their end-to-end LLM), and our 13-category F1 is a coarser score than their 55-concept macro-F1. The comparison we control cleanly is the A, B, C contrast in Table 5.

6.2 Ablations and student-level evaluation

Table 8 adds every signal subset on the same split and five seeds. Error type alone has the highest exploratory mean (32.00%), identified after inspecting the ablations and not independently validated; removing the summary from B gives 29.98%, a gain of 2.59 points. Input sizes and parameter counts also change, so these are feature-removal comparisons, not isolated causal effects.

Table 6: Per-category deficiency F1 (mean over seeds), arm B minus arm A. A positive value means the LLM signals helped that category on average. Pos. counts positive test interactions, not concept labels; zero-support categories remain in macro-F1.

Category	Pos.	A	B	Δ (B-A)
Sequences and Series	2	0.114	0.520	+0.406
Physics	6	0.214	0.246	+0.032
Statistics and Probability	8	0.000	0.044	+0.044
Ratio and Proportion	7	0.000	0.000	+0.000
Set Theory	1	0.133	0.089	-0.044
Prime Numbers	2	0.391	0.365	-0.026
Geometry	8	0.533	0.534	+0.001
Finance	8	0.446	0.438	-0.008
Polynomials	1	0.467	0.467	+0.000
Basic Arithmetic	9	0.000	0.000	+0.000
Factors and Multiples	0	0.000	0.000	+0.000
Permutations and Combinations	6	0.857	0.857	+0.000
Other	0	0.000	0.000	+0.000

Table 7: ConceptKT baselines [5], for context only, not directly comparable to Table 5.

Method	Correctness acc.	Deficiency macro-F1
DKT	64.80	n/a
DKVMN	63.10	n/a
GKT	63.20	n/a
SAKT	66.46	n/a
OKT	69.25	1.87
Best LLM (DeepSeek-R1)	71.02	17.40

Table 9 gives per-student results and six leave-one-student-out (LOSO) folds at seed 0, with the same hyperparameters. The held student never selects weights or thresholds; their past history is available at inference on the official test tail. Pooled LOSO F1@13 is A 25.70%, B 24.43%, C 31.08% (chronological seed 0: 22.93%, 23.64%, 31.14%). B’s gain does not extend to unseen students here. Folds share training students and are not independent samples.

Averaged over five seeds on the 511 associated test entries, ten-bin, equal-width ECE is .1803/.1789/.1798 for A/B/C; Brier scores are .1267/.1255/.1252. Masked zeros are excluded. These scores are not calibrated probabilities. All per-seed and LOSO thresholds and reliability bins are saved with the results.

Table 8: All signal subsets on the same five seeds and official split. M: missing concepts; E: error type; S: summary. Parameters include the allocated error embedding, even where unused.

Inputs	F1@13	SD	Δ vs A (pp)	Parameters
A	24.28	3.92	+0.00	34920
A + M	23.29	1.65	-0.98	49000
A + E	32.00	1.73	+7.73	36968
A + S	23.61	1.18	-0.67	133224
A + M + E	29.98	3.46	+5.70	51048
A + M + S	22.97	1.21	-1.31	147304
A + E + S	25.56	2.87	+1.29	135272
A + M + E + S (B)	27.39	5.03	+3.11	149352
Gold M + E (C)	32.06	1.94	+7.79	149352

Table 9: Per-student F1@13 (percent): chronological means over five seeds, LOSO at seed 0. Pos. counts concept labels.

Student	Targets	Pos.	A	B	C	LOSO A	LOSO B	LOSO C
S1	68	8	7.69	7.69	7.69	7.69	0.00	7.69
S2	68	4	1.54	1.54	2.56	0.00	0.00	0.00
S3	68	10	10.00	9.23	9.23	7.69	16.03	18.28
S4	66	8	13.53	13.42	13.40	15.85	15.85	13.29
S5	68	19	27.41	32.48	40.87	40.64	28.39	38.46
S6	66	10	18.67	17.44	17.86	19.57	19.96	16.61

7 Discussion

The answer to the research question remains exploratory. Offline LLM signals from solution steps improve mean concept deficiency prediction over correctness alone, by 3.1 points of macro-F1@13, with gains in both precision and recall and an A, B, C mean order at both levels of detail. The gain is not statistically significant and does not extend to LOSO. Full B does not exceed the mask-only baseline. The B–A gain comes from features that are read once, offline, from each past solution, and the model that uses them is small and cheap to run. That is exactly the role the proposal set out to test: the LLM as an analyzer of the process, not as the predictor.

Arm C earns its place as a yardstick rather than just a baseline. By covering about 40% of the observed A to C gap, arm B shows that the offline analyzer already captures part of the signal measured by this comparison, but the 4.7 points left over do not guarantee what a better analyzer could add. Part of that leftover gap may be down to the model we chose: we ran a small, low-cost LLM to keep the extraction cheap, and a bigger state-of-the-art model might read the solutions more accurately, but this has not been tested. The 50.6% agreement between the analyzer and the gold error types shows substantial noise. It is worth being plain about the noise. With six students and 59 positive test labels, the per-seed F1@13 moves by several points, so neither the order nor the decimals are statistically reliable across seeds. Arm C’s tighter spread (± 1.94) alone does not establish that cleaner features raise and steady the score.

8 Limitations and Future Work

The dataset is tiny, six students and 59 positive test labels, so the numbers are noisy and the conclusions point in a direction rather than pin down a value. MathEDU has no question text, so the analyzer works from the solution only, though the targets are gold and so unaffected. We use one prompt and a single small, low-cost LLM, chosen to keep costs down, with no sensitivity study, and the backbone is a plain LSTM rather than a state-of-the-art KT model.

A few next steps would help, roughly in order of value. Swapping the LSTM for a stronger backbone such as SAKT or DKVMN through pyKT [10], while keeping the same features, is the obvious first move. Replacing the plain concatenation of features with a learned gate would let the model turn down noisy analyzer signals when they are not helping. More seeds and students would strengthen the leave-one-student-out check of generalization beyond the time split. A prompt-sensitivity study would show whether the offline-feature idea is stable, which is the whole point. Joining the real question text from MathQA and re-reading would test whether more context helps the analyzer. Finally, a per-concept threshold instead of one global cutoff could squeeze out a bit more.

8.1 Threats to validity

Internal: conflicting duplicate annotations remain; validation is reused for stopping and threshold selection, and ablations are exploratory. **Construct:** sparse missing-concept labels are not a full measure of knowledge; target-associated concepts are assumed known. **External:** six students, one prompt and one LLM limit generalization; LOSO does not show a B advantage.

9 AI Usage

Gemini was the offline process analyzer. Codex and Claude helped write the code and structure the final report. Codex also helped implement, test and evaluate the revisions. Reported results come from the saved experiments with cached features.

10 Conclusion

We built a small, repeatable hybrid for concept deficiency prediction. A Gemini model reads each past solution once, offline, and writes down a few structured signals, and a small leakage-safe LSTM learns from those past-only signals to predict future weak concepts. On the official MathEDU/ConceptKT split, the LLM signals raise mean deficiency macro-F1@13 from 24.3% to 27.4% over a correctness-only baseline and cover about 40% of the observed gap to a gold-feature reference of 32.1%, with both precision and recall improving. The result comes with tested leakage checks and occurrence-specific feature keys, direct handling of the class imbalance, five-seed reporting, and offline reproducibility with the recorded cache and software environment. With very little data, turning student solution steps into simple structured signals with an offline LLM remains an exploratory approach. The gain over A is not statistically significant, does not extend to LOSO. Full B does not exceed the mask-only baseline. Error-type-only features warrant further evaluation.

References

- [1] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, “Deep Knowledge Tracing,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [2] T. Huang, X. Ou, H. Yang, Z. Long, and C. Xing, “Remembering is Not Applying: Interpretable Knowledge Tracing for Problem-Solving Processes,” in *Proc. 32nd ACM Int. Conf. on Multimedia (MM)*, 2024, <https://doi.org/10.1145/3664647.3681049>.
- [3] U. Lee, J. Bae, D. Kim, S. Lee, J. Park, T. Ahn, G. Lee, D. Stratton, and H. Kim, “Language Model Can Do Knowledge Tracing: Simple but Effective Method to Integrate Language Model and Knowledge Tracing Task,” *arXiv:2406.02893*, <https://arxiv.org/abs/2406.02893>, 2024.
- [4] W.-L. Hsu, Y.-C. Tang, and A.-Z. Yen, “MathEDU: Feedback Generation on Problem-Solving Processes for Mathematical Learning Support,” in *Proc. 19th Conf. of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026, pp. 2883–2901. <https://doi.org/10.18653/v1/2026.eacl-1.ong.132>.
- [5] Y.-C. Kang, Y.-C. Tang, and A.-Z. Yen, “ConceptKT: A Benchmark for Concept-Level Deficiency Prediction in Knowledge Tracing,” *arXiv:2603.24073* (accepted at LREC 2026), <https://arxiv.org/abs/2603.24073>, 2026.
- [6] J. Park, S. Kang, J. Park, J. Kim, J. Shin, S. Park, and Y. Yu, “Tracing Mathematical Proficiency Through Problem-Solving Processes,” *Findings of ACL*, 2026. <https://doi.org/10.18653/v1/2026.findings-acl.961>.
- [7] Y. Ozyurt, S. Feuerriegel, and M. Sachan, “Automated Knowledge Concept Annotation and Question Representation Learning for Knowledge Tracing,” *arXiv preprint arXiv:2410.01727*, 2024. <https://arxiv.org/abs/2410.01727>.
- [8] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019. <https://doi.org/10.18653/v1/D19-1410>.
- [9] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, 2017. <https://arxiv.org/abs/1708.02002>.
- [10] Z. Liu, Q. Liu, J. Chen, S. Huang, J. Tang, and W. Luo, “pyKT: A Python Library to Benchmark Deep Learning based Knowledge Tracing Models,” in *NeurIPS Datasets and Benchmarks Track*, 2022. <https://doi.org/10.52202/068431-1347>.